
Fixed Geometric Routing from Angular Structure

Casey Allard
Independent Researcher

Abstract

Sparse expert models usually obtain conditional computation by learning a gate, then regularizing that gate so routing remains trainable. This paper studies a narrower alternative: whether a fixed routing map derived from embedding geometry can provide a useful routing scaffold without learning a gate matrix. We evaluate a Hopf-derived sector map built from a fixed 4D projection of normalized representations. On a semantic proxy based on structured WikiText-2-derived embeddings, the method yields a sublinear effective routing footprint with canonical fit $\hat{e} = 2.957K^{0.572}$ over $K = 25\text{--}400$, versus linear scaling for dense routing, and the advantage persists through $K = 5000$. The proxy result is strengthened by two falsification-style controls: concentration weakens under column permutation of the embeddings, and fixed Hopf routing is more concentrated than adaptive K -means on the same proxy. In a toy two-layer language model with 64 experts, fixed Hopf routing partially transfers: on both Penn Treebank and WikiText-2 it stays within about 8% of a learned top-1 baseline while removing the learned gate matrix. The central limitation is also the central null result: in end-to-end LM training, Hopf routing is not meaningfully better than randomly permuted fixed routing. The appropriate conclusion is therefore narrow. Angular structure is sufficient to support concentrated fixed routing on a semantic proxy, and fixed routing can partially transfer to toy LM training, but the present evidence does not establish a geometry-specific quality advantage in trainable language models.

1 Introduction

Sparse routing reduces active computation, but standard mixture-of-experts systems achieve that sparsity with a learned gate and then spend additional machinery making the gate trainable and balanced (Shazeer et al., 2017; Fedus et al., 2022). That design works, but it also entangles routing quality with gate optimization. Paper 1 asks a narrower question: can some routing structure be supplied directly by the geometry of normalized representations, before learning enters the picture?

The object under test is a fixed routing map. Each input vector is normalized, projected into a fixed four-dimensional coordinate system, converted into a low-dimensional angular representation, and discretized into a routing sector. No gate matrix is learned. The motivation is empirical rather than philosophical. Prior evidence suggests that normalized language representations are anisotropic and carry angular structure rather than behaving like uniform points on the sphere (Google Research, 2026). If that structure is real, then a fixed angular partition may receive highly non-uniform traffic and therefore concentrate routing without learning a gate.

This paper makes three claims, and only those three claims. First, on a semantic proxy built from structured embeddings, fixed Hopf-derived routing yields a clearly sublinear effective routing footprint and retains that advantage through large routing budgets. Second, the same fixed scaffold partially transfers to a toy trainable language model, staying within roughly 8% of a learned top-1 baseline on Penn Treebank and WikiText-2 while removing the learned gate matrix. Third, that transfer result is sharply limited by a null finding: after end-to-end training, Hopf routing is not dis-

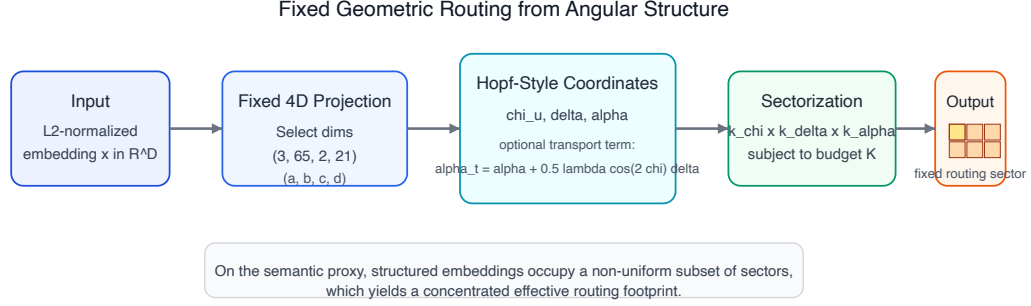


Figure 1: Fixed geometric routing in Paper 1. A normalized representation is projected onto a fixed 4D subspace, expressed in Hopf-style coordinates, optionally transport-corrected, and discretized into a sector under routing budget K . The semantic-proxy result is positive because structured embeddings occupy these sectors non-uniformly.

tinguishable from a randomly permuted fixed scaffold on quality. The paper is therefore about fixed geometric routing as a constrained first result, not about replacing learned sparse routing in general.

2 Background and Evaluation Scope

The closest point of comparison is learned sparse routing in transformer-style models (Shazeer et al., 2017; Fedus et al., 2022). Those systems optimize the routing function jointly with the experts, which makes it difficult to separate two questions: whether a routing map induces concentrated traffic, and whether the full learned system can exploit that map effectively. Paper 1 separates those questions explicitly.

The first evaluation setting is a semantic proxy built from static structured embeddings derived from WikiText-2 (Merity et al., 2016). This proxy strips away expert co-adaptation and isolates the routing map itself. It is therefore the correct place to test the core geometric hypothesis: whether angular structure alone is sufficient to induce concentrated routing. The second setting is a toy trainable language model evaluated on Penn Treebank (Marcus et al., 1993) and WikiText-2. That setting is not intended as a modern large-scale sparse-model benchmark. Instead, it asks whether the fixed scaffold remains usable after experts are trained end to end.

Across both settings, the main response variable is the *effective routing footprint*: the effective number of sectors or expert paths used under the empirical routing distribution. In the canonical analyses this quantity appears as `effective_bucket_count` on the proxy and `eff_buckets` in the trainable LM. Lower values at fixed routing budget K mean that traffic is more concentrated.

3 Fixed Geometric Routing Method

3.1 Intuition

The routing mechanism is intentionally simple. Start with a normalized representation, choose a fixed four-dimensional coordinate chart, summarize that chart with three angular coordinates, and quantize those coordinates into a finite routing budget. If the data distribution is close to uniform in those coordinates, routing will spread almost linearly with K . If the data distribution is anisotropic, traffic will collect in a smaller subset of sectors. The method is therefore a test of whether normalized language representations carry enough stable angular structure for a fixed partition to act as a useful routing scaffold.

3.2 Fixed 4D projection and Hopf coordinates

Figure 1 summarizes the mapping. Let $\mathbf{x} \in \mathbb{R}^D$ be a normalized input vector. The canonical routing rule selects a fixed coordinate tuple $(3, 65, 2, 21)$ and writes the projected coordinates as (a, b, c, d) .

The purpose of this 4D chart is practical rather than metaphysical: it creates a small coordinate system in which angular structure can be parameterized explicitly and discretized without learning a separate gating network.

From (a, b, c, d) the method computes two planar radii and three angular quantities:

$$\rho_1 = \sqrt{a^2 + b^2}, \quad \rho_2 = \sqrt{c^2 + d^2}, \quad (1)$$

$$\chi_u = \frac{\rho_2^2}{\rho_1^2 + \rho_2^2}, \quad \delta = \arctan 2(b, a) - \arctan 2(d, c), \quad (2)$$

$$\alpha = \frac{1}{2}(\arctan 2(b, a) + \arctan 2(d, c)). \quad (3)$$

These coordinates are useful because they separate three different aspects of the 4D projection. The variable χ_u records how mass splits between the two 2D planes. The variable δ records the relative angle between those planes. The variable α is a mean phase coordinate. In Paper 1, the point of this parameterization is not to claim deep geometric optimality; it is to provide a concrete, low-dimensional angular address that can be held fixed across all evaluations.

3.3 Transport-corrected phase and sectorization

The canonical routing rule includes a geometry-dependent transport correction on the phase coordinate,

$$\chi = \arcsin\left(\frac{\rho_2}{\sqrt{\rho_1^2 + \rho_2^2}}\right), \quad \alpha_t = \alpha + \frac{1}{2}\lambda \cos(2\chi) \delta, \quad (4)$$

where λ is fixed by the routing rule. Mechanically, this correction perturbs the mean phase according to where the point lies in the low-dimensional geometry. Two inputs with similar raw phase can therefore receive different sector addresses if their base coordinate and relative-angle coordinate differ. Paper 1 does not treat this transport term as an independent contribution, but it is part of the canonical routing function and helps explain why the sector map is more than a crude angular histogram.

Given routing budget K , the coordinate triple $(\chi_u, \delta, \alpha_t)$ is discretized into product bins $(k_\chi, k_\delta, k_\alpha)$ with total budget at most K . The resulting bin index is the routing address. Because the address is fixed, any reduction in routing footprint must come from the data distribution rather than from gate learning. This is the central mechanism claim of the paper: sectorization can produce concentrated routing only if normalized representations occupy these coordinates unevenly.

3.4 Paper-facing metric

The primary routing metric is the effective routing footprint. Intuitively, it measures how many routes are effectively active under the empirical occupancy distribution rather than how many routes are merely available. On the semantic proxy, the same analyses also report sector entropy and peak sector mass $p_{\max}$, which help distinguish genuine concentration from trivial occupancy effects. In the trainable LM, the same idea is carried over as an effective expert-path count.

4 Experimental Setup

4.1 Semantic proxy

The semantic proxy uses a WikiText-2-derived PPMI-SVD embedding dataset. This setting is intentionally static: there is no end-to-end language-model backpropagation and no expert co-adaptation. The routing budget is swept from $K = 25$ to $K = 400$ for the canonical scaling law and extended to $K = 5000$ for the large-budget persistence check. The main outputs are effective routing footprint, sector entropy, and peak sector mass. The role of this proxy is narrow but important. It isolates whether the routing map itself concentrates structured language embeddings more than dense routing or structure-destroying controls.

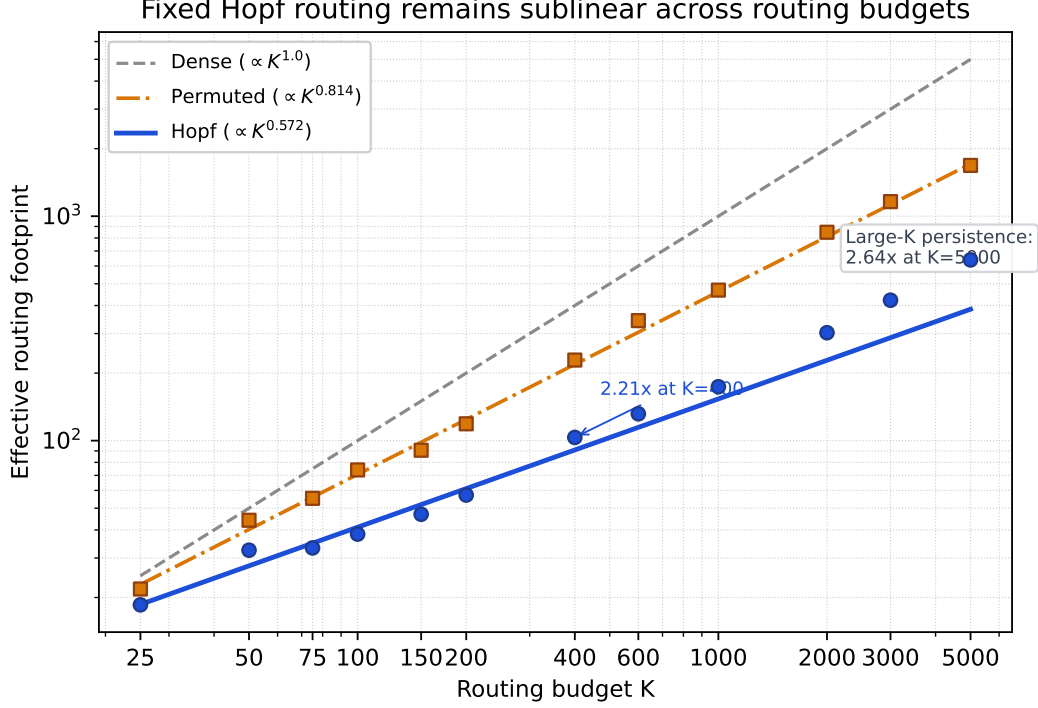


Figure 2: Primary proxy result generated directly from canonical analysis JSON. Fixed Hopf routing yields a sublinear effective routing footprint on the semantic proxy, while dense routing scales linearly and the permuted fixed-routing control scales more steeply. The large-budget extension shows that the advantage persists through $K = 5000$.

4.2 Toy language-model transfer

The trainable setting uses a two-layer transformer-like language model with hidden dimension 128, four attention heads, vocabulary size 5000, sequence length 32, 64 experts, and expert dimension 128. All conditions train for 4000 optimization steps with two seeds per dataset. The paper reports confirmatory runs on Penn Treebank and WikiText-2. The routing budget is fixed at $K = 64$ experts.

Three conditions are compared in the main text. BASELINE is learned top-1 routing without an auxiliary load-balancing loss. HOPF replaces the learned gate with the fixed geometric sector map. PERMUTED keeps the same fixed-routing scaffold but randomly permutes the sector assignments. That null condition is crucial: it preserves sparsity and parameter count while removing geometry-specific assignment. Appendix D includes the auxiliary-loss control used to justify the no-aux top-1 baseline.

5 Semantic Proxy Results

5.1 Main scaling result

The semantic proxy supplies the paper’s strongest evidence. Figure 2 shows effective routing footprint as a function of routing budget. Dense routing scales linearly by construction. Fixed Hopf routing scales much more slowly: over the canonical anchor range $K = 25$ –400, the fit is

$$\hat{e} = 2.957K^{0.572}, \quad (5)$$

while the corresponding permuted fixed-routing control scales with exponent 0.814. The effect is not confined to the anchor range. When the canonical analysis is extended to $K = 5000$, Hopf routing remains sublinear with refit exponent approximately 0.657, and the compression advantage over the permuted control reaches $2.21\times$ at $K = 400$ and $2.64\times$ at $K = 5000$.

Table 1: Main trainable-LM result. Fixed Hopf routing partially transfers to a two-layer LM on both PTB and WikiText-2, but it does not improve over the permuted fixed-routing null. Table values are generated from canonical analysis exports under paper1_assets/.

Dataset	Condition	Val. PPL	Eff. paths	64/eff _b
PTB	BASELINE	152.26	43.19	1.48×
PTB	HOPF	164.54	45.96	1.39×
PTB	PERMUTED	164.41	45.81	1.40×
WT2	BASELINE	105.85	35.37	1.81×
WT2	HOPF	114.45	40.94	1.56×
WT2	PERMUTED	114.48	42.28	1.51×

This result would be much weaker without controls, so the paper makes the most important falsifications explicit even when their full tables are deferred to the appendix. First, the concentration is sensitive to structure in the embeddings. When the same proxy embeddings are column-permuted before routing, sector $p_{\max}$ drops from 0.090 to 0.061, and sector entropy rises accordingly. Second, the effect is not reproduced equally well by a generic adaptive Euclidean comparator: on the same structured proxy, fixed Hopf routing reaches sector $p_{\max} = 0.087$, whereas adaptive K -means reaches 0.072. Appendix C gives the full tables and control figure, and Appendix C.1 shows that the scaling exponent is stable across L1, L2, L3, and L4 normalization, with span below 0.000.

6 Trainable Language-Model Results

6.1 Transfer to PTB and WT2

Table 1 summarizes the trainable LM results. On Penn Treebank, fixed Hopf routing reaches a HOPF/BASELINE validation-perplexity ratio of 1.081. On WikiText-2, the same ratio is 1.081. In both datasets, then, the fixed scaffold remains within about 8% of the learned top-1 baseline while removing the learned gate matrix entirely.

The routing-efficiency side is modest but consistent. Hopf uses 46.0 effective expert paths on PTB and 40.9 on WikiText-2 out of 64 available experts. That is not a proxy-style compression result, but it shows that the fixed scaffold can function in a trainable model without catastrophic collapse.

6.2 Explicit null result against permuted routing

The most important limitation in the manuscript is also its most scientifically useful negative result. In both datasets, HOPF and PERMUTED are effectively indistinguishable on validation perplexity: the difference is +0.13 on PTB and -0.03 on WikiText-2. Appendix D provides the compact null-result table and the full cross-dataset summary.

This null result changes the interpretation of the LM evidence. The semantic proxy shows that structured embeddings interact meaningfully with the Hopf-derived sector map. But once experts are trained end to end, they can co-adapt to whichever fixed scaffold they are given. The trainable result therefore supports a claim about fixed routing as a usable scaffold, not a claim that Hopf geometry is uniquely superior after training.

6.3 Comparator choice

The learned comparator is intentionally a top-1 gate without auxiliary load-balancing loss. That choice reflects the paper’s question: whether a fixed routing map can replace a naturally concentrated sparse regime. In the auxiliary-loss control, the balance objective drives learned routing toward near-uniform usage, with 62.8 effective paths out of 64. Appendix D includes the corresponding table.

That control is part of the evidence base, but it supports comparator choice rather than the headline result.

7 Discussion and Limitations

Paper 1 supports a bounded conclusion. The proxy experiments show that a fixed angular sector map can induce concentrated routing on structured language embeddings, and they do so with large-budget persistence and explicit falsification checks. The trainable LM experiments show that the same fixed scaffold is usable after end-to-end training on two datasets. Those are positive results.

The paper is equally defined by what it does *not* show. The strongest evidence is proxy-side rather than LM-side. The trainable architecture is toy-scale: two layers, 64 experts, and short runs. The experiments address FFN routing only, not attention routing or broader compute allocation. Most importantly, the geometry-specific advantage disappears under expert co-adaptation: Hopf routing is not better than a randomly permuted fixed scaffold after training. That negative result is substantive. It is the reason the paper argues for fixed geometric routing as a constrained first result rather than as a broad replacement for learned routing.

8 Conclusion

This paper isolates a narrow claim and supports it with a deliberately layered evidence base. A fixed Hopf-derived routing map built from normalized embedding geometry yields a sublinear effective routing footprint on a semantic proxy, retains that advantage through large routing budgets, and weakens under structure-destroying controls. The same scaffold partially transfers to a toy trainable language model without a learned gate matrix. At the same time, the trainable experiments show that Hopf geometry does not outperform a randomly permuted fixed scaffold on quality. The correct conclusion is therefore limited but real: angular structure is sufficient to support concentrated fixed routing, but the present evidence does not establish a geometry-specific advantage in end-to-end language-model training.

References

- William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23:1–39, 2022.
- Google Research. *TurboQuant*: Extreme KV-cache compression via angular rotation-invariant quantization. *arXiv preprint*, 2026.
- Mitchell P. Marcus, Mary Ann Marcinkiewicz, and Beatrice Santorini. Building a large annotated corpus of English: The Penn Treebank. *Computational Linguistics*, 19(2):313–330, 1993.
- Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. Pointer sentinel mixture models. *arXiv preprint*, 2016.
- Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarczyk, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *International Conference on Learning Representations (ICLR)*, 2017.

A Appendix

This appendix strengthens the paper’s trust surface rather than widening its claims. All tables and figures are generated from the canonical analysis files identified in `paper1_assets/asset_provenance.md`.

B Detailed Setup

Table 2: Exact LM setup values used in the canonical toy-language-model analyses.

Hyperparameter	Value
Layers	2
Hidden dimension	128
Attention heads	4
Vocabulary size	5000
Sequence length	32
Experts	64
Expert dimension	128
Training steps	4000
Datasets	Penn Treebank, WikiText-2
Conditions	BASELINE, HOPF, PERMUTED

C Semantic Proxy Controls and Robustness

Proxy controls show concentration follows structured geometry

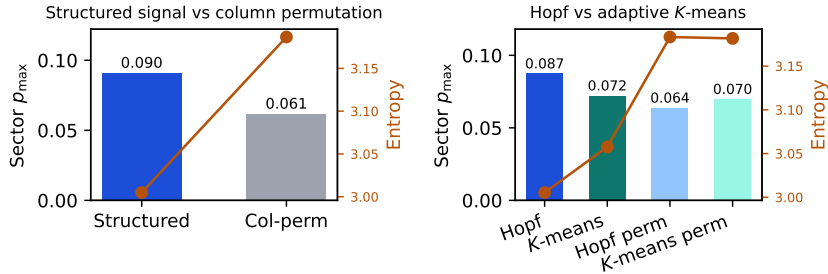


Figure 3: Proxy-side controls. Left: concentration weakens when structured embeddings are column-permuted before routing. Right: fixed Hopf routing is more concentrated than adaptive K -means on the same proxy, and both families weaken under permutation controls.

Table 3: Structured-versus-permuted proxy control. Concentration weakens when the same proxy embeddings are column-permuted before routing.

Proxy condition	Runs	Sector $p_{\max}$	Sector entropy	Test MSE
Structured	4	0.090	3.005	0.0040
Column-permuted	4	0.061	3.187	0.0040

Table 4: Hopf-versus- K -means control on the semantic proxy. Fixed Hopf routing is more concentrated than adaptive K -means on the same structured embeddings, and both weaken under permutation.

Routing family	Runs	Sector $p_{\max}$	Sector entropy	Test MSE
Hopf	2	0.087	3.005	0.0040
K -means	2	0.072	3.058	0.0040
Hopf permuted	2	0.064	3.183	0.0040
K -means permuted	2	0.070	3.182	0.0040

C.1 Normalization robustness

Table 5: Norm robustness of the proxy scaling law. The Hopf exponent remains essentially unchanged across L1, L2, L3, and L4 normalization.

Normalization	Hopf exponent	Permuted exponent	Base exponent
L1	0.572	0.809	0.916
L2	0.572	0.814	0.916
L3	0.572	0.815	0.916
L4	0.572	0.816	0.916

D Language-Model Support Tables

Table 6: Compact null-result summary for the trainable LM.

Dataset	HOPF / BASE	HOPF - PERM	HOPF eff. paths	PERM eff. paths
PTB	1.081	+0.13	46.0	45.8
WT2	1.081	-0.03	40.9	42.3

Table 7: Cross-dataset summary for the LM transfer result.

Metric	PTB	WT2
HOPF / BASELINE PPL ratio	$1.081\times$	$1.081\times$
HOPF - PERMUTED PPL	+0.13	-0.03
HOPF effective-path ratio	$1.39\times$	$1.56\times$
BASELINE eff. paths	43.2	35.4
HOPF eff. paths	46.0	40.9
PERMUTED eff. paths	45.8	42.3

Table 8: Auxiliary-loss control used to justify the main learned comparator. The learned sparse model with explicit balance loss is substantially less concentrated than the no-aux baseline.

Condition	Val. PPL	Eff. paths	Params
BASELINE	154.55	44.0	5.66M
LEARNED_SPARSE	154.78	62.8	5.66M
HOPF	165.59	47.7	5.64M
PERMUTED	165.25	47.1	5.64M

Table 9: Full control and null-result summary from the canonical generated assets.

Case	HOPF BASE	/	HOPF PERM	-	Aux paths	eff.	Interpretation
PTB	1.081		+0.13		–		Hopf and permuted are effectively indistinguishable on PTB quality.
WT2	1.081		-0.03		–		The same null result replicates on WT2.
PTB_aux_control_screen	1.071		+0.34		62.8		Load-balancing aux loss drives routing toward near-uniform usage, so no-aux top-1 is the correct paper comparator.

Table 10: PTB detail table exported from canonical analysis results.

Condition	Val. PPL	Eff. paths	64/eff _b	Params
BASELINE	152.26	43.19	1.48×	5.66M
HOPF	164.54	45.96	1.39×	5.64M
PERMUTED	164.41	45.81	1.40×	5.64M

Table 11: WikiText-2 detail table exported from canonical analysis results.

Condition	Val. PPL	Std. PPL	Eff. paths	64/eff _b
BASELINE	105.85	1.57	35.37	1.81×
HOPF	114.45	1.14	40.94	1.56×
PERMUTED	114.48	0.87	42.28	1.51×